ОТЧЕТ

По лабораторной работе №2

Выполнил:
ст. гр. 410901
Зайцев Е. А.

Условие задания:

Необходимо разработать простейший аналог утилиты traceroute / tracert, изученной в рамках первой лабораторной работы. В качестве протокола передачи данных можно использовать ICMP или UDP (на выбор). При реализации использовать программный интерфейс сокетов (готовые реализации отправки/получения echo запросов использовать запрещено).

Утилита должна принимать в качестве параметра IP-адрес целевого узла и выводить узлы маршрута аналогично системному traceroute / tracert (порядковый номер, время ожидания ответа для каждого отправленного пакета и адрес узла). Следует отправлять более одного пакета.

Проверьте, что разработанная программа дает такие же результаты, как и системная. Проанализируйте трафик, генерируемый вашей программой в процессе работы, с помощью Wireshark. Также Wireshark можно использовать при отладке, в случае проблем с получением эхо-ответов.

Обратите внимание на значение контрольной суммы (Wireshark показывает, верно ли оно подсчитано) и обновление значения sequence number между отправками (изучите, как оно изменяется в системной утилите traceroute / tracert).

ВАЖНО: сетевой экран (фаервол) может блокировать ICMP-пакеты с типом "TTL expired", получение и анализ которых необходим для определения промежуточных узлов на маршруте. Отключите сетевой экран во время тестирования или настройте таким образом, чтобы нужные пакеты не фильтровались.

Добавить параметр командной строки, активирующий режим разрешения в имена узлов через обратные DNS-запросы. В этом режиме помимо IP-адресов узлов маршрута должны выводиться и их имена (такое поведение является поведением по умолчанию для системных tracert / traceroute).

Добавить возможность задавать имя целевого узла вместо его IP-адреса: mytraceroute google.com.

```
D:\КПО\курсовая\coursework\KsisLab2\KsisLab2>dotnet run -- 8.8.8.8

Трассировка маршрута к 8.8.8.8 [8.8.8.8]
с максимальным числом прыжков 30:

  1   15ms     1ms     0ms    192.168.10.1
  2    3ms     2ms     2ms    212.98.160.250
  3    1ms     1ms     3ms    212.98.160.168
  4    7ms     3ms    12ms    185.11.76.41
  5    8ms     2ms     2ms    185.11.76.26
  6   11ms    12ms     9ms    74.125.49.252
  7   26ms    28ms    34ms    108.170.234.167
  8   26ms    30ms    29ms    142.250.224.89
  9   27ms    30ms    29ms    8.8.8.8

Трассировка завершена.

D:\КПО\курсовая\coursework\KsisLab2\KsisLab2>
```

Рисунок 2 – Результат выполнения утилиты для 8.8.8.8

Описание: на представленном изображении показан маршрут к IP-адресу 8.8.8.8. Здесь изображены все промежуточные узлы, через которые проходит пакет. Как и для tracert, для более достоверной работы утилиты выполняется 3 соответствующих запроса. Слева мы видим порядкой номер (ttl для каждого запроса), также отображена задержка получения ответа с каждого узла. Также видим IP-адреса всех промежуточных узлов. В конце получаем сообщение об окончании трассировки.

Теперь рассмотрим результат работы в Wireshark:

| | | | | | | |
|---|---|---|---|---|---|---|
| 12 | 4.890374 | 192.168.10.105 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x4a3f, seq=0/0, ttl=1 (no response found!) |
| 13 | 4.903997 | 192.168.10.1 | 192.168.10.105 | ICMP | 102 | Time-to-live exceeded (Time to live exceeded in transit) |
| 14 | 4.906900 | 192.168.10.105 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x4a3f, seq=1/256, ttl=1 (no response found!) |
| 15 | 4.907786 | 192.168.10.1 | 192.168.10.105 | ICMP | 102 | Time-to-live exceeded (Time to live exceeded in transit) |
| 16 | 4.907935 | 192.168.10.105 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x4a3f, seq=2/512, ttl=1 (no response found!) |
| 17 | 4.908742 | 192.168.10.1 | 192.168.10.105 | ICMP | 102 | Time-to-live exceeded (Time to live exceeded in transit) |
| 18 | 4.909339 | 192.168.10.105 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x4a3f, seq=3/768, ttl=2 (no response found!) |
| 19 | 4.911853 | 212.98.160.250 | 192.168.10.105 | ICMP | 110 | Time-to-live exceeded (Time to live exceeded in transit) |
| 20 | 4.911955 | 192.168.10.105 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x4a3f, seq=4/1024, ttl=2 (no response found!) |
| 21 | 4.914006 | 212.98.160.250 | 192.168.10.105 | ICMP | 110 | Time-to-live exceeded (Time to live exceeded in transit) |
| 22 | 4.914186 | 192.168.10.105 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x4a3f, seq=5/1280, ttl=2 (no response found!) |
| 23 | 4.921568 | 212.98.160.250 | 192.168.10.105 | ICMP | 110 | Time-to-live exceeded (Time to live exceeded in transit) |
| 24 | 4.921844 | 192.168.10.105 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x4a3f, seq=6/1536, ttl=3 (no response found!) |
| 25 | 4.923653 | 212.98.160.168 | 192.168.10.105 | ICMP | 110 | Time-to-live exceeded (Time to live exceeded in transit) |
| 26 | 4.923766 | 192.168.10.105 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x4a3f, seq=7/1792, ttl=3 (no response found!) |
| 27 | 4.931823 | 212.98.160.168 | 192.168.10.105 | ICMP | 110 | Time-to-live exceeded (Time to live exceeded in transit) |
| 28 | 4.931987 | 192.168.10.105 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x4a3f, seq=8/2048, ttl=3 (no response found!) |

Рисунок 3 – Результат работы Wireshark

Также в конце мы можем увидеть 3 запроса echo request и 3 соответствующих echo reply, что говорит нам об успешном достижении целевого узла

| | | | | | | |
|---|---|---|---|---|---|---|
| 87 | 5.152529 | 192.168.10.105 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x4a3f, seq=24/6144, ttl=9 (reply in 88) |
| 88 | 5.181741 | 8.8.8.8 | 192.168.10.105 | ICMP | 74 | Echo (ping) reply id=0x4a3f, seq=24/6144, ttl=114 (request in 87) |
| 89 | 5.182188 | 192.168.10.105 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x4a3f, seq=25/6400, ttl=9 (reply in 91) |
| 91 | 5.209648 | 8.8.8.8 | 192.168.10.105 | ICMP | 74 | Echo (ping) reply id=0x4a3f, seq=25/6400, ttl=114 (request in 89) |
| 92 | 5.210101 | 192.168.10.105 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x4a3f, seq=26/6656, ttl=9 (reply in 93) |
| 93 | 5.237911 | 8.8.8.8 | 192.168.10.105 | ICMP | 74 | Echo (ping) reply id=0x4a3f, seq=26/6656, ttl=114 (request in 92) |

Рисунок 4 – три соответствующих echo request/reply

На рисунке показаны ICMP-пакеты типа «Echo request», отправленные с текущего узла, а также ICMP-пакеты типа «Echo reply», полученные от узла назначения.

```
▸ Frame 44: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 b
▸ Ethernet II, Src: AzureWaveTec_99:68:6a (f8:54:f6:99:68:6a), Dst: CudyT
▸ Internet Protocol Version 4, Src: 192.168.10.105, Dst: 8.8.8.8
▾ Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0x2026 [correct]
    [Checksum Status: Good]
    Identifier (BE): 19007 (0x4a3f)
    Identifier (LE): 16202 (0x3f4a)
    Sequence Number (BE): 16 (0x0010)
    Sequence Number (LE): 4096 (0x1000)
  ▸ [No response seen]
  ▸ Data (32 bytes)
```

Рисунок 5 – Заголовки echo request

```
▸ Frame 88: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface \Device\NPF_{07E8A
▸ Ethernet II, Src: CudyTechnolo_5d:65:5c (80:af:ca:5d:65:5c), Dst: AzureWaveTec_99:68:6a (f8:54:f6:99:68:6a)
▸ Internet Protocol Version 4, Src: 8.8.8.8, Dst: 192.168.10.105
▾ Internet Control Message Protocol
    Type: Echo (ping) reply (0)
    Code: 0
    Checksum: 0x281e [correct]
    [Checksum Status: Good]
    Identifier (BE): 19007 (0x4a3f)
    Identifier (LE): 16202 (0x3f4a)
    Sequence Number (BE): 24 (0x0018)
    Sequence Number (LE): 6144 (0x1800)
    [Request frame: 87]
    [Response time: 29,212 ms]
  ▸ Data (32 bytes)
```

Рисунок 6 – Заголовки echo reply

```
▸ Frame 53: Packet, 70 bytes on wire (560 bits), 70 bytes captured (560 bi
▸ Ethernet II, Src: CudyTechnolo_5d:65:5c (80:af:ca:5d:65:5c), Dst: AzureW
▸ Internet Protocol Version 4, Src: 74.125.49.252, Dst: 192.168.10.105
▾ Internet Control Message Protocol
  ▸ Type: Time-to-live exceeded (11)
    Code: 0 (Time to live exceeded in transit)
    Checksum: 0x828a [correct]
    [Checksum Status: Good]
    Unused: 00000000
  ▸ Internet Protocol Version 4, Src: 192.168.10.105, Dst: 8.8.8.8
  ▾ Internet Control Message Protocol
      Type: Echo (ping) request (8)
      Code: 0
      Checksum: 0x2025 [unverified] [in ICMP error packet]
      [Checksum Status: Unverified]
      Identifier (BE): 19007 (0x4a3f)
      Identifier (LE): 16202 (0x3f4a)
      Sequence Number (BE): 17 (0x0011)
      Sequence Number (LE): 4352 (0x1100)
```

Рисунок 7 – Заголовки time-to-live exceeded

Теперь выполним tracert и сравним результат работы:

```
D:\КПО\курсовая\coursework\KsisLab2\KsisLab2>tracert 8.8.8.8

Трассировка маршрута к dns.google [8.8.8.8]
с максимальным числом прыжков 30:

  1    <1 мс    <1 мс    <1 мс  192.168.10.1
  2    15 ms     1 ms     3 ms  asr-dc.bn.by [212.98.160.250]
  3    21 ms     1 ms     2 ms  msq.asr-mck.bn.by [212.98.160.168]
  4     2 ms     3 ms     4 ms  185.11.76.41
  5    17 ms     2 ms     3 ms  185.11.76.26
  6    21 ms    21 ms     9 ms  74.125.49.252
  7    37 ms    29 ms    26 ms  108.170.234.167
  8    41 ms    30 ms    27 ms  142.250.224.89
  9    40 ms    27 ms    27 ms  dns.google [8.8.8.8]

Трассировка завершена.

D:\КПО\курсовая\coursework\KsisLab2\KsisLab2>_
```

Рисунок 8 – Результат выполнения tracert для того же IP-адреса

```
▸ Frame 99: Packet, 106 bytes on wire (848 bits), 106 bytes captured (848 bits) on in
▸ Ethernet II, Src: AzureWaveTec_99:68:6a (f8:54:f6:99:68:6a), Dst: CudyTechnolo_5d:6
▸ Internet Protocol Version 4, Src: 192.168.10.105, Dst: 8.8.8.8
▾ Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0xf62f [correct]
    [Checksum Status: Good]
    Identifier (BE): 1 (0x0001)
    Identifier (LE): 256 (0x0100)
    Sequence Number (BE): 463 (0x01cf)
    Sequence Number (LE): 52993 (0xcf01)
  ▸ [No response seen]
  ▸ Data (64 bytes)
```

Рисунок 9 – Заголовки echo request при выполнении tracert

Как мы видим, созданная в ходе лабораторной работы утилита отправляет icmp-запросы и строит маршрут до нужного узла.

Следующим этапом будем запускать разработанную утилиту для домена. Для примера, будем брать домен yandex.ru. Принцип работы аналогичен прошлому примеры. Единственное отличие заключается лишь в том, что сначала будет отправляться DNS-запрос, который позволит узнать IP-адрес необходимого узла. А далее программа проделает такой же алгоритм работы.

```
D:\КПО\курсовая\coursework\KsisLab2\KsisLab2>dotnet run -- yandex.ru

Трассировка маршрута к 77.88.44.55 [77.88.44.55]
с максимальным числом прыжков 30:

  1   11ms     1ms     3ms    192.168.10.1
  2   45ms    11ms    18ms    212.98.160.250
  3    7ms     1ms     1ms    212.98.160.168
  4   15ms    14ms    10ms    185.11.76.41
  5    5ms     6ms     6ms    185.11.76.26
  6   24ms    12ms    12ms    185.11.76.87
  7    *       *       *      Превышен интервал ожидания для запроса.
  8    *       *       *      Превышен интервал ожидания для запроса.
  9   28ms    18ms    18ms    93.158.160.212
 10   16ms    16ms    16ms    10.32.57.1
 11   27ms    16ms    16ms    10.1.7.1
 12   16ms    16ms    16ms    77.88.44.55

Трассировка завершена.
```

Рисунок 10 – Результат работы утилиты для домена

```
D:\КПО\курсовая\coursework\KsisLab2\KsisLab2>tracert yandex.ru

Трассировка маршрута к YANDEX.ru [77.88.44.55]
с максимальным числом прыжков 30:

  1    13 ms     1 ms     1 ms  192.168.10.1
  2    53 ms     7 ms     6 ms  asr-dc.bn.by [212.98.160.250]
  3     7 ms     4 ms     4 ms  msq.asr-mck.bn.by [212.98.160.168]
  4     3 ms     2 ms     2 ms  185.11.76.41
  5    27 ms     2 ms     4 ms  185.11.76.26
  6    25 ms    12 ms    12 ms  185.11.76.87
  7     *        *        *     Превышен интервал ожидания для запроса.
  8     *        *        *     Превышен интервал ожидания для запроса.
  9    24 ms    18 ms    26 ms  klg-32z5-ae4.yndx.net [93.158.160.212]
 10    16 ms    16 ms    16 ms  10.32.57.1
 11    29 ms    26 ms    19 ms  10.1.7.1
 12    28 ms    17 ms    16 ms  yandex.ru [77.88.44.55]

Трассировка завершена.
```

Рисунок 11 – Результат работы tracert для того же домена

Как мы видим, разработанная утилита и tracert показали аналогичный результат работы. Также обратим внимание на результат работы Wireshark, а именно на наличие DNS-запроса, необходимого для получения IP-адреса.

| | | | | | | |
|---|---|---|---|---|---|---|
| 24 | 4.231170 | 192.168.10.105 | 192.168.10.1 | DNS | 69 | Standard query 0x567a A yandex.ru |
| 25 | 4.240641 | 192.168.10.1 | 192.168.10.105 | DNS | 124 | Standard query response 0x567a A yandex.ru A 77. |

Рисунок 12 – DNS-запросы в Wireshark

Таким образом, работа утилиты для домена также корректна.

Теперь рассмотрим также возможность использовать флаг «-use». Он позволяет активировать режим разрешения в имена узлов через обратные DNS-запросы. В этом режиме помимо IP-адресов узлов маршрута должны выводиться и их имена. Для tracert такое поведение установлено по умолчанию.

```
D:\КПО\курсовая\coursework\KsisLab2\KsisLab2>dotnet run -- -use 8.8.8.8

Трассировка маршрута к dns.google [8.8.8.8]
с максимальным числом прыжков 30:

  1    2ms     1ms     0ms    192.168.10.1
  2    2ms     1ms     2ms    asr-dc.bn.by [212.98.160.250]
  3    3ms     2ms     2ms    msq.asr-mck.bn.by [212.98.160.168]
  4    2ms     1ms     6ms    185.11.76.41
  5    7ms     2ms     2ms    185.11.76.26
  6   11ms    13ms     9ms    74.125.49.252
  7   28ms    26ms    29ms    108.170.234.167
  8   34ms    30ms    26ms    142.250.224.89
  9   36ms    33ms    42ms    dns.google [8.8.8.8]

Трассировка завершена.
```

Рисунок 13 – Результат работы программы с флагом для 8.8.8.8

Данный флаг отправляет PTR-запросы, которые позволяют узнать домен. Как мы видим, в начале выполнения теперь отображается домен. Далее для всех узлов, которые имеют домен, он также отображается и таким же образом получается из PTR-запроса. Сравним с tracert.

```
D:\КПО\курсовая\coursework\KsisLab2\KsisLab2>tracert 8.8.8.8

Трассировка маршрута к dns.google [8.8.8.8]
с максимальным числом прыжков 30:

  1    16 ms     6 ms     1 ms  192.168.10.1
  2    29 ms     4 ms    14 ms  asr-dc.bn.by [212.98.160.250]
  3     2 ms     3 ms     1 ms  msq.asr-mck.bn.by [212.98.160.168]
  4     3 ms     2 ms     4 ms  185.11.76.41
  5    26 ms    13 ms     2 ms  185.11.76.26
  6    25 ms     9 ms     9 ms  74.125.49.252
  7    31 ms    26 ms    27 ms  108.170.234.167
  8    27 ms    26 ms    26 ms  142.250.224.89
  9    43 ms    27 ms    33 ms  dns.google [8.8.8.8]

Трассировка завершена.

D:\КПО\курсовая\coursework\KsisLab2\KsisLab2>_
```

Рисунок 14 – Результат работы tracert с флагом для 8.8.8.8

Как мы видим, программа с флагом «-use» работает аналогично tracert – она также вывела домены и корректно прошла весь маршрут. В Wireshark мы также можем увидеть отправленные PTR-запросы.

| | | | | |
|---|---|---|---|---|
| 192.168.10.105 | 192.168.10.1 | DNS | 85 | Standard query 0xbca1 PTR 41.76.11.185.in-addr.arpa |
| 192.168.10.1 | 192.168.10.105 | DNS | 139 | Standard query response 0xbca1 No such name PTR 41.76.11.185.in-addr.arpa S |
| 192.168.10.105 | 192.168.10.1 | DNS | 85 | Standard query 0x869b PTR 26.76.11.185.in-addr.arpa |
| 192.168.10.1 | 192.168.10.105 | DNS | 139 | Standard query response 0x869b No such name PTR 26.76.11.185.in-addr.arpa S |
| 192.168.10.105 | 192.168.10.1 | DNS | 86 | Standard query 0xaaa6 PTR 252.49.125.74.in-addr.arpa |
| 192.168.10.105 | 192.168.10.1 | DNS | 86 | Standard query 0xaaa6 PTR 252.49.125.74.in-addr.arpa |
| 192.168.10.1 | 192.168.10.105 | DNS | 165 | Standard query response 0xaaa6 No such name PTR 252.49.125.74.in-addr.arpa |
| 192.168.10.105 | 192.168.10.1 | DNS | 88 | Standard query 0xd075 PTR 167.234.170.108.in-addr.arpa |
| 192.168.10.105 | 192.168.10.1 | DNS | 88 | Standard query 0xd075 PTR 167.234.170.108.in-addr.arpa |
| 192.168.10.1 | 192.168.10.105 | DNS | 148 | Standard query response 0xd075 No such name PTR 167.234.170.108.in-addr.arp |
| 192.168.10.105 | 192.168.10.1 | DNS | 87 | Standard query 0x58fc PTR 89.224.250.142.in-addr.arpa |
| 192.168.10.105 | 192.168.10.1 | DNS | 87 | Standard query 0x58fc PTR 89.224.250.142.in-addr.arpa |
| 192.168.10.1 | 192.168.10.105 | DNS | 167 | Standard query response 0x58fc No such name PTR 89.224.250.142.in-addr.arpa |

Рисунок 15 – PTR-запросы в Wireshark

Вывод: в ходе выполнения лабораторной работы был разработан аналог утилиты tracert. Первое что было реализовано это стандартный запуск по IP-адресу, где в терминале мы видим только IP-адреса узлов, а Wireshark показал пакеты ICMP с типами 0 (reply), 8 (request), 11 (time-to-live exceed). Далее была добавлена возможность выполнять программу не только для IP, но и для домена. Для этого использовался DNS-запрос, который помогает узнать IP, а далее программа выполнялась как и в предыдущем случае. После был реализован специальный флаг «-use», активирующий режим разрешения в имена узлов через PTR-запросы. Программа отображает в консоли доменные имена для тех узлов, которые их имеют (как и классический tracert). В ходе тестирования и сравнения с Wireshark и tracert можно сделать вывод, что программа выполняет поставленную задачу. Таким образом, программа корректно реализовывает tracert и имеет специальный флаг для активации режима разрешения имен.